Problem Statement:

Singular Value Decomposition and Image Compression

Objectives

- **Mathematical Formulation:** Decomposing high-dimensional matrix representations.
- **Low-Rank Approximation:** Compressing digital images while preserving core visual features.
- **Noise Reduction:** Reconstructing matrices to filter image artifacts and noise.

PRESENTED BY

Project Authors

Vaani Sharma

Umang Vijay

Suyash Pandey

Srinidhi Rangarajan

Introduction

Singular Value Decomposition (SVD)

Mathematical Foundation and Applications

Project Overview

- **Mathematical Foundation:** Understanding the mathematical foundation of SVD.
- **Geometric Interpretation:** Exploring the geometric interpretation of SVD.
- **Image Compression:** Applying SVD to image compression.
- **Noise Filtering:** Using SVD for noise filtering and image denoising.

OBJECTIVE

To understand how SVD decomposes a matrix and how this technique is used in image processing.

Key focus: Unlocking matrix factorization principles for real-world digital image computation.

Mathematics + SVD

Singular Value Decomposition (SVD)

Breaking a matrix into three simpler matrices

$$A = U\Sigma V^T$$

The fundamental SVD equation

U

Left singular vectors

Σ

Singular values

V^T

Right singular vectors, transposed

WHY SVD?

It provides a useful way to analyze, approximate, and process matrices.

Mathematics + SVD

The Four Key Matrices in SVD Formula: $A = U \times \Sigma \times V^T$

A (Original)
Dim: m × n

Represents: Matrix decomposed

Property: Holds original data or transformation

U (Left Singular)
Dim: m × m

Represents: Output directions

Property: Orthogonal columns (left singular vectors)

Σ (Singular Values)
Dim: m × n

Represents: Scaling factors

Property: Diagonal matrix in descending order

V^T (Right Transposed)
Dim: n × n

Represents: Input directions

Property: Orthogonal transposed right singular vectors

HOW SVD WORKS & WHY IT MATTERS

V^T (Orient): Determines input directions | **Σ (Scale):** Stretches or compresses data | **U (Transform):** Maps to output directions

Why It Matters: Identifies key components for efficient image compression and noise filtering.

Mathematics + SVD

Demonstrating SVD Using a Matrix

Original Matrix A

$$A = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix}$$

SVD: $A = U \times \Sigma \times V^T$

Singular Values & Sigma Matrix

Singular Values

$\sigma_1 = 2.303$
$\sigma_2 = 1.303$

Sigma Matrix (Σ)

$$\begin{bmatrix} 2.303 & 0 \\ 0 & 1.303 \end{bmatrix}$$

Python Implementation `U, S, VT = np.linalg.svd(A)`
U: Left singular vectors **Σ (S):** Diagonal matrix of singular values **V^T:** Transposed right singular vectors

Mathematics + SVD

Verification of SVD

$$U \times \Sigma \times V^T = A$$ ✓ VERIFIED: TRUE

1. Matrix Reconstruction

Equation: $U \times \Sigma \times V^T = A$

The reconstructed matrix perfectly matches the original input matrix.

2. Orthogonality Checks

$U^T \times U = I$ | $V^T \times V = I$

Both orthogonality conditions evaluate to **TRUE**.

Why Is SVD Important?

- Singular values indicate the relative significance of components.
- Truncated SVD approximates matrices and images using fewer components.
- **Essential foundational technique for image compression and noise filtering.**

Geometric Visualization

What does SVD mean geometrically?

SVD decomposes a transformation into three steps.

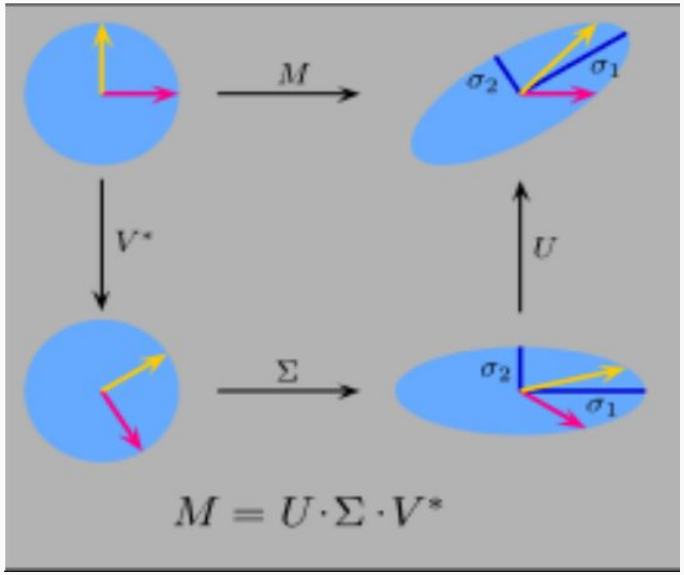
$A=U\Sigma V^T$

V^T = Rotation/Reflection

Σ = Scaling/Stretching

U = Rotation/Reflection

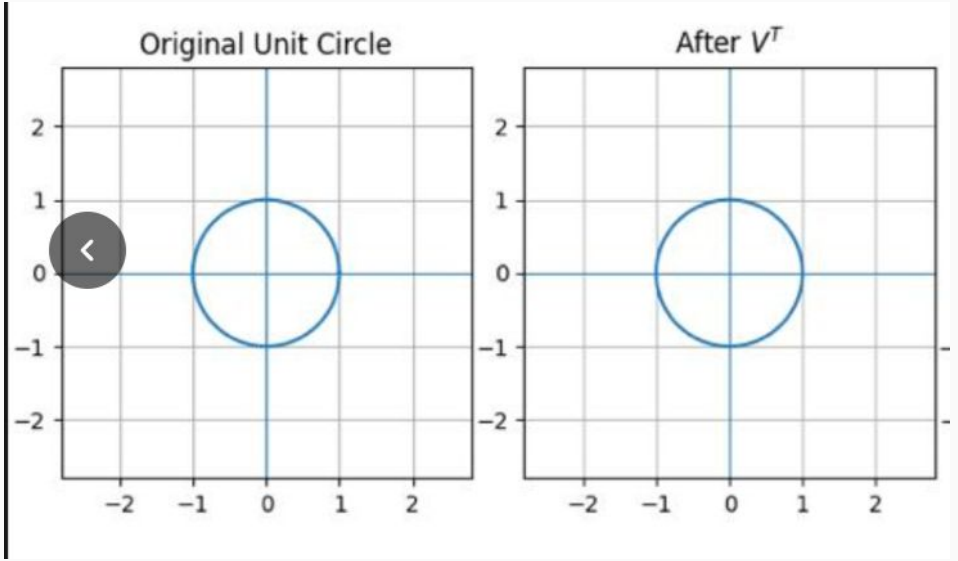
Unit Circle $\rightarrow V^T \rightarrow \Sigma \rightarrow U \rightarrow$ Ellipse



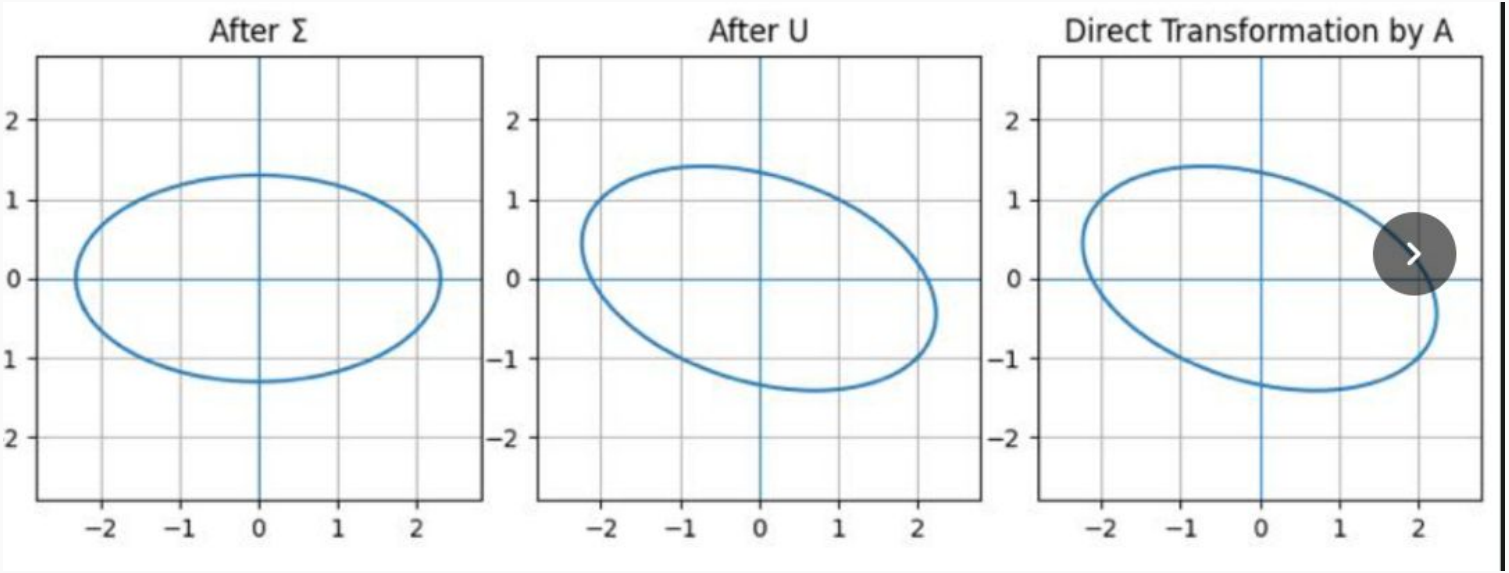
Geometric Visualization

Our code produces all the stages.

1. Original Unit Circle
2. After V^T
3. After Σ
4. After U
5. Direct transformation by A



Geometric Visualization



Intro SVD for Image Compression

Goal: reduce image data while retaining important visual information.

Method: Singular Value Decomposition with truncated reconstruction.

Values tested: k = 50, 100 and 150 singular values.

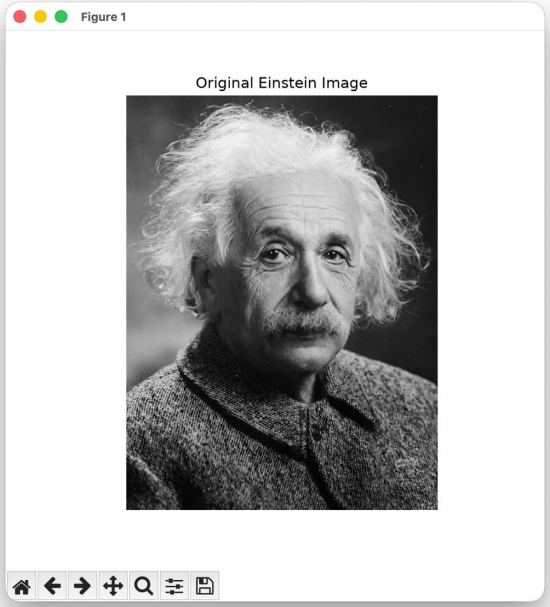
Image → Matrix

A grayscale image is represented as a matrix of pixel-intensity values.

Each matrix entry represents the brightness of a pixel.

The image matrix is decomposed using $A = U\Sigma V^T$.

Σ contains singular values arranged from largest to smallest.



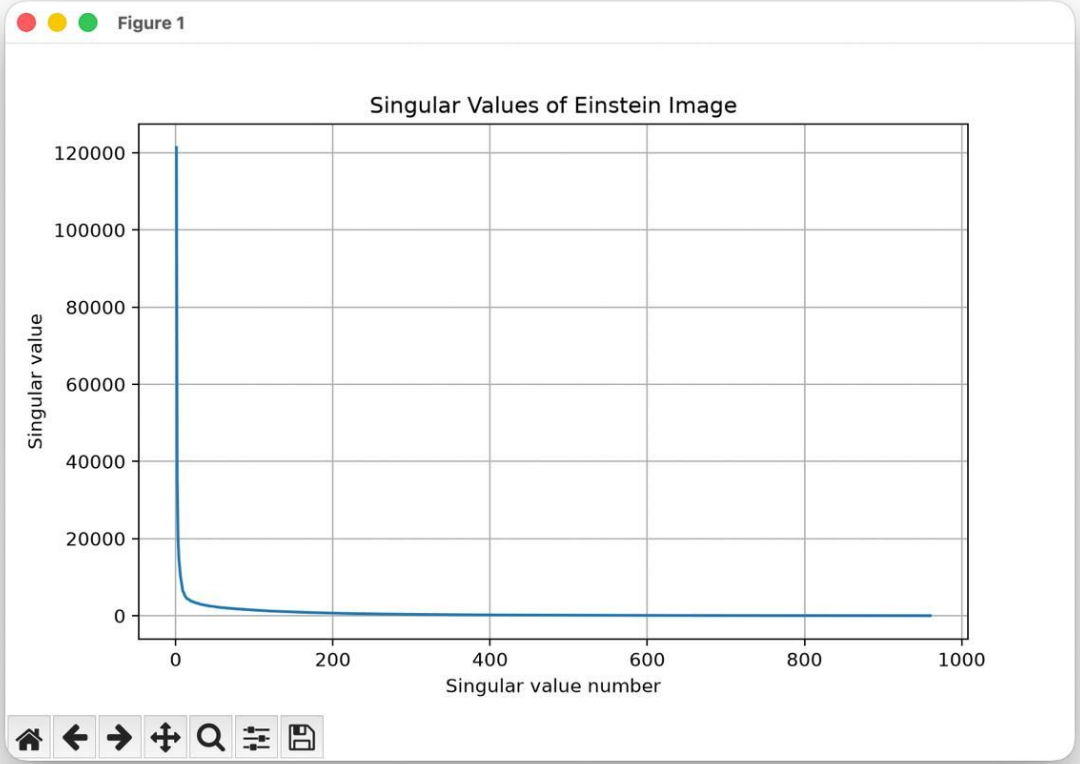
Applying SVD

NumPy computes U, Σ and V^T using np.linalg.svd().

The largest singular values contain the most significant information.

Only the first k singular values are retained.

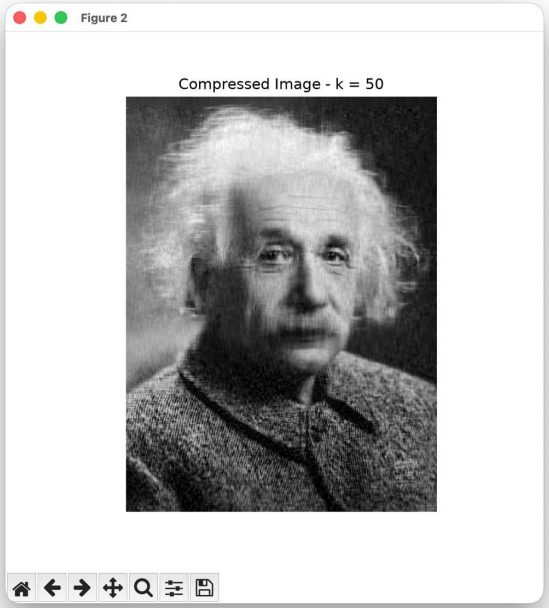
Truncated reconstruction: $A_k \approx U_k \Sigma_k V_k^T$.

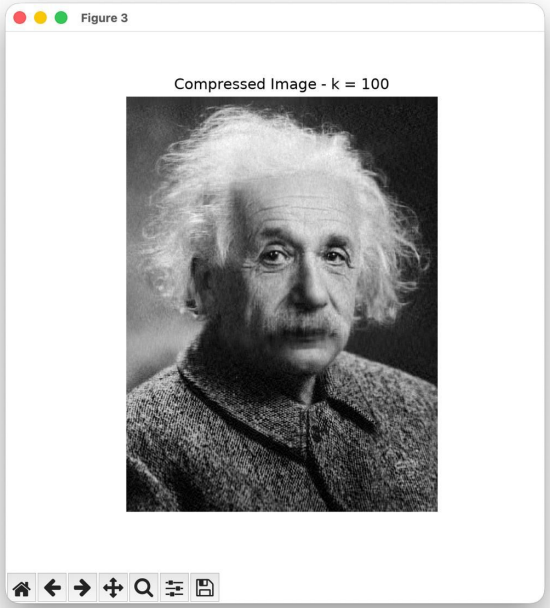


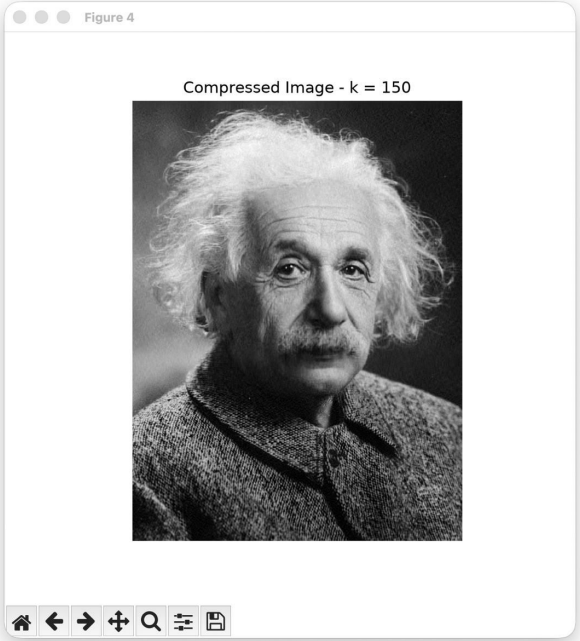
Compression Results

Smaller k → greater compression, but more information is discarded.

Larger k → better reconstruction, but less compression.







Compression Percentage & Outcome

Original storage: m × n values.

Truncated SVD storage: m × k + k × n values.

Compression % = [1 − (mk + kn)/(mn)] × 100.

Outcome: the image is represented using fewer values while preserving its main visual structure.

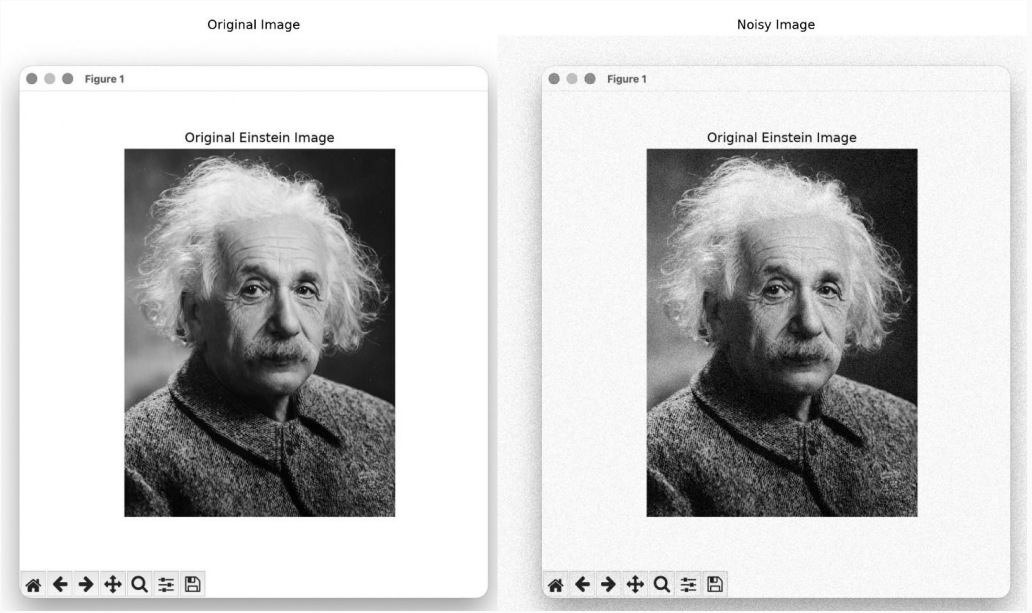
Concept → Purpose → Outcome: SVD → dimensionality reduction → compressed image.

```
(.venv) umangvijay@Umangs-MacBook-Air SVD_project % python person3_image_compression.py
Image matrix size: 1280 x 960
Number of singular values: 960
----------------------------------------
k = 50
Original values stored: 1228800
Compressed values stored: 112000
Compression percentage: 90.89 %
----------------------------------------
k = 100
Original values stored: 1228800
Compressed values stored: 224000
Compression percentage: 81.77 %
----------------------------------------
k = 150
Original values stored: 1228800
Compressed values stored: 336000
Compression percentage: 72.66 %
(.venv) umangvijay@Umangs-MacBook-Air SVD_project %
```

Noise Generation

Demonstrates how noise affects an image.

- Loaded the original **`checkers.pgm`** as a grayscale image.
- Generated random noise using NumPy.
- Added noise in the range of approximately −25 to +25 to each pixel.
- Clipped pixel values to the valid range of 0–255.



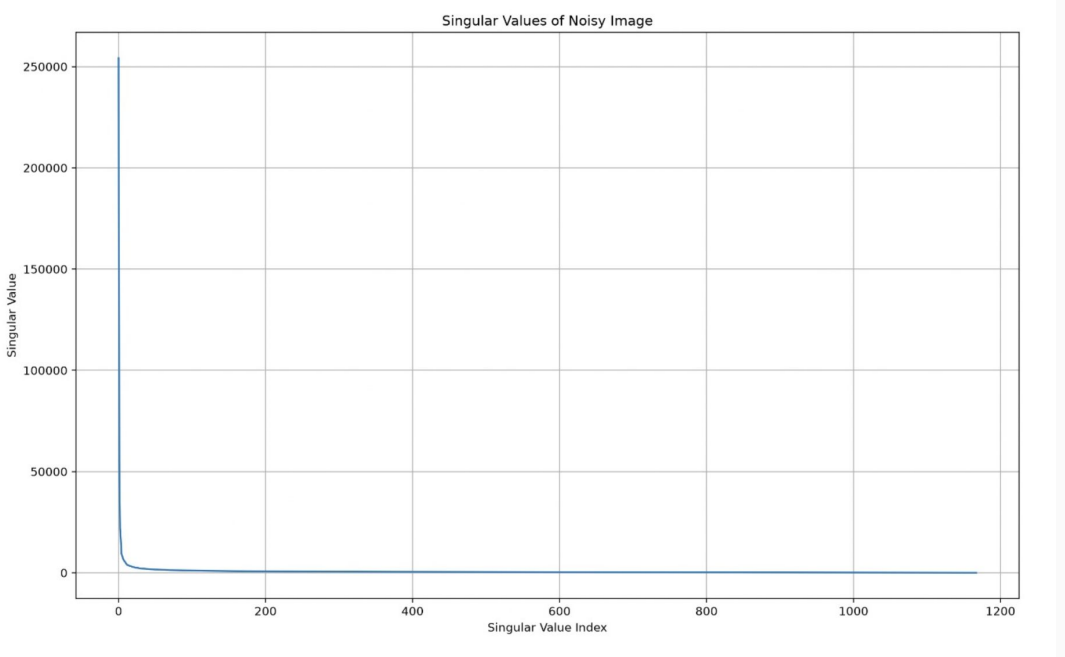
SVD of the Noisy Image

Decompose the noisy image into its fundamental components.

- Applied Singular Value Decomposition (SVD) to the noisy image.
- Decomposed the image matrix as $A=U\Sigma V^T$
- Singular values represent the relative importance of image components.
- Retaining dominant components helps preserve major image structures.

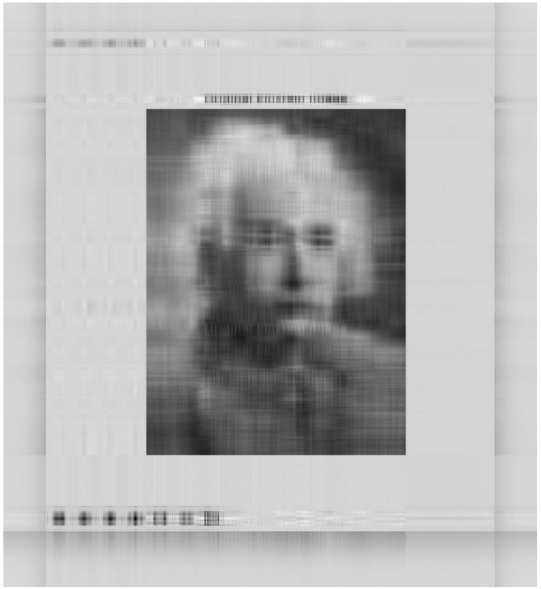
```
Image loaded successfully!
Image dimensions: (1280, 1168)

SVD completed!
U dimensions: (1280, 1168)
Number of singular values: 1168
V dimensions: (1168, 1168)
```

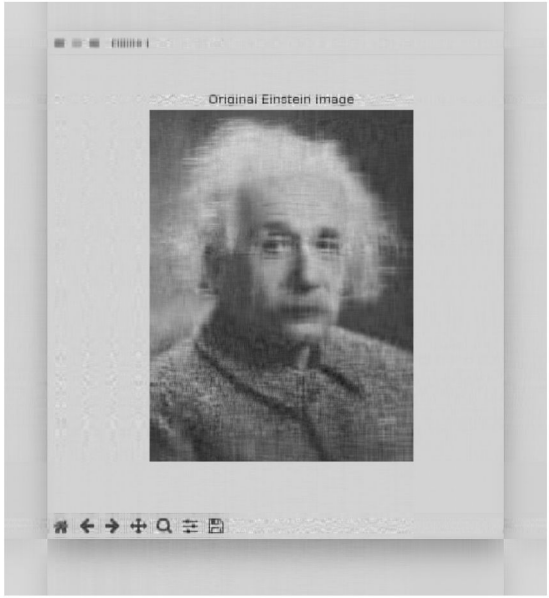


SVD-Based Denoising

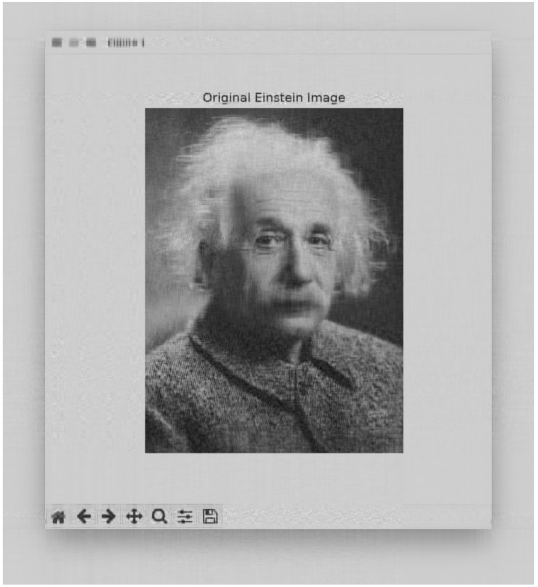
Denoised Image - k = 10



Denoised Image - k = 30



Denoised Image - k = 50



Results & Comparison for SVD-Based Denoising

Objective: Reduce noise using truncated SVD.

- Reconstructed the image using only the first (k) singular values.
- Tested three values: (k = 10, 30, 50).

Key Observations

- The noisy image contains random pixel variations.
- (k = 10) produces the smoothest, blurriest reconstruction.
- (k = 30) and (k = 50) retain progressively more image detail.
- SVD demonstrates the trade-off between noise reduction and image sharpness.